

Jasper: Engineering a Global Workspace for Latent Reasoning in Hybrid Retention Models

Jasper Research Project

August 2026

Abstract

Recent work from Anthropic has revealed that language models develop a compact, privileged internal workspace—the J-SPACE—that causally carries multi-step reasoning, while Samsung’s Tiny Recursive Model (TRM) has demonstrated that iterative refinement through a tiny recurrent network can substitute for parameter count on hard reasoning tasks. We present Jasper, an architecture that combines these two insights into a single system: a hybrid retention/attention backbone with an explicitly engineered workspace module and a recurrent core that iterates on the workspace state. The key architectural observation is that the retention layer’s fixed-size compressed state is functionally shaped like the J-SPACE—a coincidence that makes the combination more powerful than applying the same ideas to a pure transformer. The retention layer (decayed linear attention) preserves the key property—a fixed-size, input-dependent state—while being simple and fast on the target hardware. We argue that this approach is a direct and more efficient alternative to chain-of-thought “thinking tokens”: it reasons in latent space without generating tokens, keeps memory flat in both sequence length and loop count, and allows trading inference-time compute depth for parameter count. We describe a proof-of-concept experiment that trains three parameter-matched model variants on synthetic reasoning tasks with controllable depth, and outline the pre-registered criteria for a go/no-go decision on scaling.

Contents

1	Introduction	3
2	Background	3
2.1	The J-Space: A Global Workspace in Language Models	3
2.2	Iterative Reasoning with Tiny Networks	4
2.3	Retention: Decayed Linear Attention	4
3	The Retention Coincidence	4
4	Architecture	5
4.1	Overview	5
4.2	Workspace Module	5
4.3	Recurrent Core	7
4.4	Hybrid Backbone	7
5	Comparison to Thinking Tokens	7

6	Experimental Design	8
6.1	Ablation Ladder	8
6.2	Training Configuration	8
6.3	Synthetic Tasks	11
6.4	Pre-Registered Results	11
7	Preliminary Results	12
8	Discussion	13
8.1	Why This Might Work	13
8.2	Why It Might Not Work	13
8.3	Decision Framework	14
9	Related Work	14
10	Conclusion	15

1 Introduction

Large language models have demonstrated impressive reasoning capabilities, but the dominant paradigm for scaling reasoning—generating long chains of “thinking tokens” before producing an answer—has fundamental inefficiencies. Each thinking token requires a full forward pass through the model, the key-value cache grows linearly with the length of the reasoning trace, and the entire chain must be serialized in token space, consuming both memory and latency. On constrained deployment targets such as web browsers or mobile devices, this memory growth is the binding constraint.

Two recent findings suggest an alternative path. First, Anthropic’s discovery of the J-SPACE (Anthropic, 2026) revealed that the reasoning-critical machinery inside a language model is already small: a compact set of verbalizable representations, occupying less than 10% of activation variance, carries the causal bulk of multi-step reasoning. This workspace is not the full residual stream—it is a privileged, broadcasted subset that functions analogously to the global workspace in cognitive science (Baars, 2005). Second, Samsung’s Tiny Recursive Model (TRM) (Jolicoeur-Martineau, 2025) and Geiping et al.’s recurrent-depth transformers (Geiping et al., 2025) have shown that iterative refinement through a small recurrent network can substitute for parameter count: a 7M-parameter model recursing on itself can outperform models with 1000× more parameters on reasoning benchmarks.

Jasper combines these insights. We engineer an explicit workspace module—a compact set of learned slot vectors that serve as a persistent scratchpad—and pair it with a recurrent core that iterates on the workspace state. The backbone is a hybrid retention/attention model. The retention layer (decayed linear attention (Sun et al., 2024)) maintains a fixed-size, input-dependent compressed state—the key property that makes it functionally shaped like the J-SPACE. The architecture already has a mechanism that resembles the global workspace, so the engineered workspace and the retention state reinforce each other rather than competing for the same functional role.

The result is a model that reasons in latent space without generating tokens, keeps memory flat in both sequence length and loop count, and can trade inference-time compute (more loop iterations) for parameter count. We argue that this is a direct alternative to thinking tokens—one that is more memory-efficient, faster, and potentially more capable of reasoning that is not easily expressed in words.

2 Background

2.1 The J-Space: A Global Workspace in Language Models

Anthropic (2026) introduced the Jacobian lens (J-lens), an interpretability technique that identifies which concepts a language model is *poised to verbalize* at any point in its processing. Applying the J-lens across layers reveals a set of representations—the J-SPACE—that exhibits the functional properties of a global workspace (Dehaene, 2014; Baars, 2005):

- **Verbal report:** J-space contents can be read out as words the model could say.
- **Directed modulation:** Instructing the model to think about a concept causes that concept to appear in the J-space.
- **Silent reasoning:** Intermediate steps of multi-step problems appear in the J-space before the model verbalizes them.
- **Selective causality:** Ablating the J-space destroys multi-step reasoning while leaving single-hop recall intact.
- **Compactness:** The J-space carries fewer than ~25 concepts at a time and occupies <10% of activation variance.

Crucially, the J-SPACE was not designed—it emerged during training. But its properties are

exactly what one would want from an engineered reasoning workspace: it is small, privileged, broadcasted, and causally responsible for multi-step reasoning. This raises the question: *can we engineer such a workspace explicitly, and does doing so improve reasoning in a small model?*

2.2 Iterative Reasoning with Tiny Networks

Jolicoeur-Martineau (2025) introduced the Tiny Recursive Model (TRM), a 7M-parameter network that recurses on itself to iteratively refine answers. TRM alternates between a “think” step (updating a latent scratchpad $\mathbf{z}$) and an “act” step (updating a solution embedding $\mathbf{y}$), unrolled for up to 16 iterations with deep supervision. Despite having less than 0.01% of the parameters of frontier LLMs, TRM achieves 45% accuracy on ARC-AGI-1 and 8% on ARC-AGI-2, outperforming models like DeepSeek R1 and o3-mini on these benchmarks.

Independently, Geiping et al. (2025) demonstrated that recurrent-depth transformers—which iterate a transformer block for a variable number of steps at inference time—can scale test-time compute in latent space. Their 3.5B-parameter model, trained on 800B tokens, improves performance on reasoning benchmarks with additional inference-time iterations, sometimes matching the performance of a 50B-parameter model.

Both works share a key insight: *depth (iterations) can substitute for width (parameters)*. But both use attention-based cores, which means the key-value cache grows with sequence length, and the recurrent state is carried in the residual stream—a high-dimensional, unstructured representation.

2.3 Retention: Decayed Linear Attention

Sun et al. (2024) introduced the Retentive Network (RetNet), which replaces standard attention’s softmax normalization with a decayed linear attention formulation. The retention layer computes:

$$y_t = \sum_{s \leq t} \gamma^{t-s} \cdot (C_t^\top B_s) \cdot V_s \quad (1)$$

where B_s, C_t are input-dependent projections, V_s is the value vector, and $\gamma \in (0, 1)$ is a learned per-head decay scalar. This is equivalent to attention with a learned exponential decay mask $D[t, s] = \gamma^{t-s}$ in place of softmax normalization.

The key property for our purposes is that the retention layer maintains a **fixed-size state** $\mathbf{h}_t$ that is updated at each position:

$$\mathbf{h}_t = \gamma \mathbf{h}_{t-1} + B_t^\top V_t \quad (2)$$

This state is **compressed**: it has a fixed dimension regardless of sequence length, and it is **input-dependent**: the projections B_t, V_t are computed from the current input. This is functionally analogous to the J-SPACE—a small, privileged state that carries task-relevant information forward through the computation. The decay γ provides a natural forgetting mechanism: old information fades unless actively reinforced by new inputs, preventing unbounded state growth.

The retention formulation is pure matrix multiplications (no selective scan or causal conv1d), which makes it well-suited to the Tenstorrent Blackhole hardware. The backward pass (gradient of the loss w.r.t. γ) is computed via a row-sum/col-sum reduction of the $T \times T$ decay matrix, on-device.

3 The Retention Coincidence

The central architectural observation behind Jasper is what we call the *retention coincidence*: the retention layer’s compressed state already maintains a representation with the functional

shape of the J-SPACE, without any engineering. Consider the parallels:

Property	J-space (Anthropic)	Retention State
Fixed size	~ 25 concepts, $< 10\%$ of activation variance	$d_{\text{state}} = 64$ per head, regardless of sequence length
Selective	Broadcasted more widely than other representations	Input-dependent projections control write/forget
Persistent	Carries intermediate reasoning steps across layers	Carries information across the sequence
Causally responsible	Ablation destroys multi-step reasoning	Removing state connections breaks long-range dependencies
Emerges in training	Not designed, emerged in Claude	Not designed for reasoning, emerged from language modeling

Table 1: Functional parallel between the J-SPACE and the retention state.

This parallel is not merely metaphorical. When we build a hybrid model with retention layers and an explicit workspace module, the retention state and the workspace slots *reinforce each other*: the retention state provides a natural substrate for the workspace to read from and write to, and the workspace provides a structured, interpretable interface to the retention layer’s otherwise opaque state. In a pure transformer, the workspace must carve out its role from the residual stream against the competing pressure of attention’s global mixing. In a hybrid retention model, the workspace aligns with an existing architectural feature.

The coincidence becomes powerful when combined with a recurrent core. In a recurrent transformer (Geiping et al., 2025), each iteration processes the full sequence with attention, growing the KV cache. In a recurrent retention model, each iteration updates the retention state in-place—memory is flat in both sequence length *and* loop count. The workspace state, carried between iterations, provides a stable anchor for iterative refinement without any memory growth.

4 Architecture

4.1 Overview

The Jasper architecture combines four components:

1. **Retention layers** for sequence processing with flat memory. The retention layer uses decayed linear attention with a learned per-head decay scalar γ , maintaining a fixed-size compressed state while being pure matmuls.
2. **Attention layers** at selected positions for exact retrieval capability.
3. **Workspace module** — a perceiver-style cross-attention block with learned slot vectors.
4. **Recurrent core** — a range of layers that are iterated K times, with the workspace state carried between iterations.

4.2 Workspace Module

The workspace module is an explicit implementation of the J-SPACE concept. It maintains $m = 16$ learned slot vectors $\mathbf{S} \in \mathbb{R}^{m \times d}$, where $d = 384$ is the model dimension. At each application, it performs two phases:

Read phase (sequence $\rightarrow$ workspace). Slots attend over the hidden states $\mathbf{X} \in \mathbb{R}^{T \times d}$:

$$\mathbf{Q}_s = \mathbf{S} W_Q, \quad \mathbf{K}_x = \mathbf{X} W_K, \quad \mathbf{V}_x = \mathbf{X} W_V \quad (3)$$

$$\mathbf{A}_{\text{read}} = \text{softmax}\left(\frac{\mathbf{Q}_s \mathbf{K}_x^\top}{\sqrt{d_h}}\right) \quad (4)$$

$$\mathbf{S}' = \text{SlotNorm}(\mathbf{S} + \sigma(g_r) \cdot \mathbf{A}_{\text{read}} \mathbf{V}_x W_O) \quad (5)$$

where $d_h = d/n_{\text{heads}}$ is the per-head dimension, σ is the sigmoid function, and g_r is a learned scalar gate. The sigmoid gating allows the model to learn how much of the sequence to absorb into the workspace at each step.

Write phase (workspace $\rightarrow$ sequence). Hidden states attend over the updated slots:

$$\mathbf{Q}_x = \mathbf{X} W'_Q, \quad \mathbf{K}_s = \mathbf{S}' W'_K, \quad \mathbf{V}_s = \mathbf{S}' W'_V \quad (6)$$

$$\mathbf{A}_{\text{write}} = \text{softmax}\left(\frac{\mathbf{Q}_x \mathbf{K}_s^\top}{\sqrt{d_h}}\right) \quad (7)$$

$$\mathbf{X}' = \text{RMSNorm}(\mathbf{X} + \sigma(g_w) \cdot \mathbf{A}_{\text{write}} \mathbf{V}_s W'_O) \quad (8)$$

The slot state $\mathbf{S}'$ is persistent: it is carried forward to the next workspace application and, in the recurrent core, across loop iterations. SlotNorm (RMSNorm applied to the slot dimension) prevents unbounded growth of slot activations across iterations.

Slot parameter normalization. The learned slot parameters $\mathbf{S}$ are subject to a positive feedback loop: large slots produce sharp attention distributions, which produce large gradients, which cause the slots to grow further. To break this loop, the slot parameters are normalized to unit RMS after each optimizer step:

$$\mathbf{S} \leftarrow \frac{\mathbf{S}}{\text{RMS}(\mathbf{S})}, \quad \text{RMS}(\mathbf{S}) = \sqrt{\frac{1}{md} \sum_{i,j} S_{ij}^2} \quad (9)$$

This is a parameter constraint (analogous to weight clipping in GANs) rather than a learned normalization. It allows the *direction* of each slot vector to change freely while constraining its magnitude, preventing the runaway growth that causes gradient explosion.

Spectral normalization of workspace weights. Even with slot normalization, the attention weight matrices (W_Q, W_K, W_V, W_O) can grow during training, producing large attention logits that create sharp softmax distributions. To address this root cause directly, we cap the spectral norm of all eight workspace weight matrices at a bound C after each optimizer step:

$$W \leftarrow W \cdot \min\left(1, \frac{C}{\sigma_{\max}(W)}\right) \quad (10)$$

where $\sigma_{\max}(W)$ is the largest singular value, estimated via 10 steps of power iteration on-device. This is the technique that stabilized GAN discriminators Miyato et al. (2018), which face the same problem: unbounded weights $\rightarrow$ sharp distributions $\rightarrow$ gradient instability. The cap (rather than a hard normalization to C) lets the optimizer grow weights naturally up to the bound, then prevents further growth — breaking the positive feedback loop without disrupting training.

With $C = 5$, unit-RMS slots, and LayerNorm on the input, the attention logits are bounded by:

$$|\text{logit}_{ij}| \leq \|W_Q\|_2 \|W_K\|_2 \|\text{slots}\| \|x\| / \sqrt{d_h} \leq C^2 / \sqrt{d_h} \approx 2.55 \quad (11)$$

giving a maximum attention ratio of $e^{2 \times 2.55} \approx 164:1$ — sufficient for selective attention, but preventing the e^{100+} ratios that cause gradient explosion. The model can still learn the *direction* of each weight matrix (which determines what it attends to) and grow its magnitude up to C (which determines attention sharpness), but cannot grow beyond C .

Design rationale. The workspace is deliberately small ($m = 16$ slots, $\sim 2\text{M}$ parameters total) to mirror the compactness of the J-SPACE. The two-phase design (read then write) mirrors the global workspace theory’s broadcast cycle: information is first consolidated into the workspace, then broadcast back to the processing stream.

4.3 Recurrent Core

Layers c_{start} through $c_{\text{end}} - 1$ (by default, layers 6–9) form the recurrent core. These layers are applied K times in sequence, with the workspace state carried between iterations:

$$\mathbf{x}^{(k+1)}, \mathbf{S}^{(k+1)} = \text{CoreLayers}(\mathbf{x}^{(k)}, \mathbf{S}^{(k)}), \quad k = 0, \dots, K - 1 \quad (12)$$

During training, K is sampled uniformly from $\{1, \dots, K_{\text{max}}\}$ for each batch. For hardware compilation (e.g., on Tenstorrent accelerators), the loop is always unrolled to K_{max} , with iterations beyond the sampled K masked to identity via a blending operation:

$$\mathbf{x}^{(k+1)} = \alpha_k \cdot \mathbf{x}_{\text{new}}^{(k)} + (1 - \alpha_k) \cdot \mathbf{x}^{(k)} \quad (13)$$

where $\alpha_k = 1$ if $k < K$ and $\alpha_k = 0$ otherwise. This keeps the computation graph static while allowing variable-depth training.

$1/\sqrt{K}$ residual scaling. When the core layers are shared across K iterations, gradients from all iterations accumulate on the shared parameters. Without normalization, the accumulated gradient norm scales as $\sim \sigma\sqrt{K}$, causing instability at large K . To normalize this, the blend factor is scaled by $1/\sqrt{K}$:

$$\alpha_k = \frac{\mathbb{I}[k < K]}{\sqrt{K}} \quad (14)$$

This turns the full replacement ($\alpha_k = 1$) into a partial update, where each iteration contributes $1/\sqrt{K}$ of the new state. The total variance added to the signal after K iterations is then independent of K , following the variance-preserving initialization principle used in DeepNorm (Wang et al., 2022). The scaling uses the actual K sampled for each batch, so batches with different K values produce gradients of comparable magnitude.

At inference time, K can be chosen freely—this is the test-time compute scaling mechanism. Harder problems can be allocated more iterations, directly trading compute for accuracy.

4.4 Hybrid Backbone

The full model uses 13–14 layers total, with attention inserted at positions 5 and 10. The attention layers provide exact retrieval capability—a known weakness of pure retention models (Arora et al., 2024)—while the retention layers provide efficient sequence processing with flat memory. The hybrid ratio ($\sim 85\%$ retention, $\sim 15\%$ attention) follows the empirical finding that a minority of attention layers recovers most of the retrieval gap while preserving the throughput and memory advantages.

5 Comparison to Thinking Tokens

The dominant approach to scaling reasoning in language models is chain-of-thought (CoT) prompting: the model generates a sequence of intermediate “thinking tokens” before producing its final answer. This approach, while effective, has several structural inefficiencies that the workspace-recurrent architecture addresses directly.

The key distinction is that thinking tokens reason in *token space*—the model must verbalize each intermediate step, serialize it, and re-process it on the next forward pass. This is slow (one

Property	Thinking Tokens (CoT)	Workspace Recurrence (Jasper)
Reasoning medium	Token space (discrete, serial)	Latent space (continuous, parallel)
Memory growth	Linear in reasoning length (KV cache)	Flat (fixed workspace + retention state)
Tokens generated	One per reasoning step	Zero—reasoning is internal
Legibility	Fully legible (readable text)	Partially legible (workspace probes)
Compute control	Number of generated tokens	Number of loop iterations K
Training data	Requires CoT supervision or RL	Standard next-token prediction + deep supervision
Expressive range	Limited to verbalizable reasoning	Can capture non-verbal reasoning patterns
Deployment cost	High (long generation, large cache)	Low (short output, fixed memory)

Table 2: Comparison of thinking tokens vs. workspace recurrence.

forward pass per token), memory-hungry (the KV cache grows with each token), and constrains reasoning to what can be expressed in language. As Geiping et al. (2025) note, “a substantial amount of thought happens through complex, recurrent firing patterns in the brain, before the first word of an answer is uttered.”

Workspace recurrence reasons in *latent space*—the workspace state is updated in-place through matrix operations, without generating any tokens. This is faster (one forward pass per iteration, not per token), memory-flat (the workspace has a fixed size regardless of reasoning depth), and can capture reasoning patterns that are not easily expressed in words. The trade-off is legibility: thinking tokens produce readable traces, while workspace reasoning is only partially legible through probes (see Section 6).

For deployment on constrained devices (web browsers, mobile phones), the memory advantage is decisive. A model that generates 1000 thinking tokens accumulates a KV cache of $\sim 1000 \times d_{\text{model}} \times n_{\text{layers}}$ floats. A model that iterates its workspace 100 times uses the same fixed workspace state ($m \times d$ floats) plus the retention state ($d_{\text{state}} \times d$ floats), regardless of K .

6 Experimental Design

6.1 Ablation Ladder

We train three parameter-matched model variants ($\sim 10\text{M}$ parameters each) that form an ablation ladder—each cell adds one component, so the marginal contribution of each is isolated:

Cell history. The design was refined from an initial four-cell grid to the current three-cell ladder after preliminary experiments showed that the pure-backbone and high-LR variants plateaued on Task 1 without contributing additional signal. The comparison logic is: **B vs A** isolates the workspace effect (same $\text{LR}=2 \times 10^{-4}$); **C vs B** isolates the recurrent core; **C vs A** is the full R1 test.

6.2 Training Configuration

All cells use AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}$) with weight decay 0.1 and gradient clipping at norm 1.0. All cells use peak $\text{lr}=2 \times 10^{-4}$ with a linear warmup over 200 steps, followed by a constant peak. The training budget is 10,000 steps per cell. Training runs on a Tenstorrent Quietbox 2 ($4 \times$ Blackhole chips, bfloat16-native) with all three cells in parallel on separate devices.

Batch configuration. Each step uses micro-batch size 8 with 48 gradient accumulation steps, giving an effective batch size of 384 sequences ($\sim 49\text{K}$ tokens per step). Fresh data is sampled

Cell	Architecture	What it tests
A	Hybrid (14 retention + 2 attention at positions 5, 10), no workspace	The no-workspace control; isolates the workspace effect
B	Hybrid + workspace (13 retention + 2 attention + 16-slot perceiver workspace)	Does an engineered workspace help without recurrence?
C	Full architecture (hybrid + workspace + layers 6–9 looped K times)	The go/no-go cell; does recurrence + workspace beat the control?

Table 3: The three-cell ablation ladder. All cells target $\sim 10\text{M}$ parameters ($d_{\text{model}} = 384$, 13–14 layers). Cell B removes one retention layer to compensate for the $\sim 2\text{M}$ workspace parameters, keeping cells parameter-matched.

every batch—with generated data there is no train set to overfit.

Early stopping. An EMA-smoothed loss plateau detector stops training if the loss does not improve by a relative 0.1% over 1,000 steps (10% of the training budget). The EMA smoothing factor is $\beta = 0.99$, giving a half-life of ~ 69 steps ($\sim 3,300$ micro-batches), which is well above the per-step noise floor at this batch size.

Per-parameter LR groups. The workspace parameters ($\sim 2\text{M}$ of $\sim 10\text{M}$ total) receive $0.25\times$ the base learning rate (5×10^{-5} vs 2×10^{-4} for the backbone). This is motivated by the observation that the workspace creates a sharper loss landscape: at equal LR, workspace gradients dominate and cause instability. The backbone LR is kept at 2×10^{-4} to match Cell A (the control), preserving the clean B-vs-A comparison. Gate parameters (read/write gates) receive $10\times$ the workspace LR (2×10^{-3}) so they open fast enough to contribute within the training budget.

Seven stabilization techniques. The workspace and recurrent core introduce several sources of gradient instability that are not present in the baseline hybrid model. We address each with a targeted normalization:

1. **Per-parameter LR groups** (above): workspace parameters receive $0.25\times$ the base LR, preventing the workspace from dominating the gradient.
2. **$1/\sqrt{K}$ residual scaling** (Section 4.3): scales the recurrent core’s blend factor by $1/\sqrt{K}$, normalizing gradient accumulation across K iterations so that the total gradient norm is independent of K .
3. **Slot parameter normalization** (Section 4.2): constrains the learned slot parameters to unit RMS after each optimizer step, breaking the positive feedback loop where large slots $\rightarrow$ sharp attention $\rightarrow$ large gradients $\rightarrow$ larger slots.

4. **Spectral normalization** (Section 4.2): caps the spectral norm of all eight workspace weight matrices at $C = 5$ after each optimizer step, addressing the root cause of unbounded weight growth directly while preserving sufficient attention selectivity (max ratio $\sim 164:1$).
5. **Retention γ clamping**: clamps the retention layer’s log-decay scalar to ≤ -0.02 (i.e. $\gamma \leq 0.98$) after each optimizer step. When γ drifts to 1.0, the decay matrix $D[t, s] = \gamma^{t-s}$ becomes 1 everywhere—no decay—and the γ gradient (an $O(T^2)$ sum over $T \times T$ terms weighted by $\text{diff}[t, s]$) becomes $\sim 128\times$ larger than weight matrix gradients ($O(T)$ sums). This gradient dominates the global clip, starving all other parameters of update signal. The -0.02 clamp ensures $D[127] = 0.98^{127} = 0.077$, attenuating the γ gradient by $\sim 10\times$.
6. **Retention γ LR group**: the γ parameters receive $0.01\times$ the base LR, compensating for the $O(T^2)$ vs $O(T)$ gradient scale mismatch so that γ updates at a comparable effective rate to weight matrices.
7. **Skip-on-spike**: if the pre-clip gradient norm exceeds 500, the optimizer step is skipped entirely. This prevents explosion from corrupting the model—when one parameter dominates the gradient norm, clipping scales *all* gradients by $1/\text{norm}$, starving every other parameter. Skipping preserves the model state and lets the next batch’s gradient make progress. The threshold of 500 tolerates the control cell’s natural grad norm ($\sim 245\text{--}395$ from token embeddings) while catching genuine explosions ($25,000+$ and 10^{14} observed in preliminary runs).

Each technique targets a distinct mechanism: LR groups address the workspace’s sharper loss landscape, $1/\sqrt{K}$ scaling addresses recurrent gradient accumulation, slot normalization addresses unbounded slot parameter growth, spectral normalization addresses unbounded weight matrix growth, γ clamping addresses the retention decay’s structural gradient scale mismatch, the γ LR group compensates for the same mismatch at the optimizer level, and skip-on-spike provides a safety net against residual explosions. All seven are deterministic (two hyperparameters: the spectral norm bound $C = 5$, the skip threshold 500) and can be disabled independently for ablation.

Custom fused kernels. The Tenstorrent implementation uses three custom tt-metal kernels to reduce op dispatch overhead and eliminate intermediate DRAM writes: (1) fused RoPE (full rotation in a single kernel pass, using a split-RoPE optimization that avoids sub-tile concat/slice when $d_{\text{head}}/2$ is not tile-aligned), (2) fused scale+decay (scores = $\text{qk} \cdot \text{scale} \cdot D_{\text{decay}}$ in one pass), and (3) fused gate backward (computing `grad_out_flat` and `grad_g` in a single kernel pass). The retention layer runs at 2.9 ms per forward+backward pass at $d_{\text{model}} = 384$, $n_{\text{heads}} = 4$, $T = 128$, $B = 8$.

Attention regularizers (implemented, disabled). To counter workspace degeneracy we implemented an attention entropy penalty and a slot diversity penalty (`ws_entropy_weight`, `ws_diversity_weight`). Both are **disabled**, at weight 0. The entropy penalty is disabled because entropy measures *peakedness*, not *input-dependence*: the cheapest way to minimize attention entropy is a fixed one-hot routing that ignores the input entirely, which is precisely the degenerate solution the regularizer was intended to prevent. The code paths are retained for reproducibility. Selectivity is now measured, not optimized: see the input-dependence diagnostic below.

Selectivity and decoding diagnostics. Two measurements replace the discarded entropy objective. **Input-dependence**: probe the model with $N = 16$ distinct inputs of identical token length and compute the mean total variation distance between each input’s attention distribution and the across-input mean, holding the position index fixed, plus the fraction of positions whose top-1 target is identical across all probes. A fixed routing scores $\text{TV} = 0$ and 100% agreement no matter how peaked it is, so this cannot be gamed by sharpening. Reported

per path (read and write) because a functioning read path can otherwise mask a degenerate write path. **Teacher-forced answer accuracy:** evaluate answer tokens given the gold prefix, alongside free generation, to separate reasoning failure from exposure bias.

Architectural stability: zero-init gates and slot decay. The external constraints and regularizers address symptoms of instability but not the root cause: the workspace’s feedback loop is amplifying rather than contractive. Two architectural changes make the workspace stable by construction. **Zero-initialized gates:** gate parameters are initialized to -2 ($\sigma(-2) \approx 0.12$), so the workspace starts with a small contribution and the feedback loop is mostly inactive. A gate-specific LR group ($10\times$ workspace LR) ensures the gates open within the training budget. **Slot decay:** the slot update includes a learned decay factor γ (initialized at 1.0): $\text{slots} = \text{norm}(\gamma \cdot \text{slots} + g_{\text{read}} \cdot \text{read_out})$. The decay provides a restoring force—old information fades unless actively reinforced—making the feedback loop contractive. Config: `gate_init` (default -2), `slot_decay_init` (default 1.0).

6.3 Synthetic Tasks

All tasks are character-level, generated on-the-fly (infinite data, no overfitting), and automatically verifiable. Each has a **depth knob** that controls how many reasoning steps a correct answer requires.

Task 1: Chained assignment arithmetic (multi-hop composition). Variables are defined in terms of previous variables, all arithmetic mod 97:

`a=7;b=a*3+2;c=b-a;d=c*2;?d;`

The model must follow a chain of k dependencies. Distractor variables are inserted at random positions. This is the core multi-hop reasoning task.

Task 2: Permutation tracking (state-tracking). n items undergo k swap operations; the model must report one item’s final position:

`n=6;2,5;0,3;5,1;?3;`

State-tracking is a documented weakness of linear attention models (Arora et al., 2024); this task tests whether the workspace compensates for the architecture’s known gap.

Task 3: Single-hop recall (control). Shallow lookup with distractors:

`a=5;b=12;c=8;d=3;e=15;?c;`

This is the control for selective ablation: the J-SPACE signature requires that killing the workspace leaves this intact while Tasks 1–2 collapse.

Training uses depths 2–8; evaluation extends to 2–16, measuring extrapolation beyond the training distribution.

6.4 Pre-Registered Results

Four measurements constitute proof, in ascending order of importance:

R1 — Capability: Cell C beats Cell A by ≥ 10 accuracy points on Tasks 1–2 at depths 10–16 (beyond training range).

R2 — Compute scaling: Accuracy on deep problems increases monotonically with K ; harder depths need higher K to saturate.

R3 — Linear probes on workspace slots can decode intermediate variable values, with decodability rising across loop iterations. This is the J-lens analogue.
Workspace decodability:
R4 — Selective ablation: Replacing workspace slots with their training-set mean causes Tasks 1–2 to collapse (≥ 30 pt drop) while Task 3 barely moves (≤ 5 pt drop)—mirroring Anthropic’s J-SPACE finding.

R3 and R4 are what make this a *J-SPACE proof* rather than just another architecture ablation. R4 in particular tests the causal claim: the workspace is not merely correlated with reasoning performance, it is *causally responsible* for it.

7 Preliminary Results

The current training run (August 2026) uses the retention-based backbone on Tenstorrent Blackhole hardware with the three-cell grid (A, B, C) described above. All three cells are training in parallel on separate devices with a 10,000-step budget and early stopping.

Current run status. As of this writing, the three cells are at steps 150 (A), 150 (B), and 50 (C) of 10,000—a fresh run restarted after the gradient stability fixes described above. Loss is dropping steadily from the initial ~ 5.0 : Cell A at 1.42, Cell B at 1.42, Cell C at 1.88. Cell C is $\sim 2.7\times$ slower per step (11.5s vs 4.3s) due to the recurrent core’s $K_{\max} = 6$ iterations. All three cells are training stably with no gradient explosions and no skipped steps.

A critical bug was discovered and fixed early in the run: the optimizer was performing AdamW updates entirely in bf16, which froze any parameter whose magnitude was $O(1)$ or larger. The per-step update ($\sim 10^{-4}$) is far below the bf16 ULP at those magnitudes ($\sim 8 \times 10^{-3}$ at 1.0, $\sim 1.6 \times 10^{-2}$ at 2.0), so the update rounded to zero and never accumulated. This affected the workspace gates (init -2.0 , sigmoid ≈ 0.12), slot decay (init 1.0), and all RMSNorm scale weights (init 1.0)—15 frozen parameters per layer plus 14 workspace parameters. The workspace was completely inert: gates stayed at -2.0 , slots never decayed, norms never adapted. The fix is the standard mixed-precision training pattern: maintain fp32 master copies of all parameters, compute AdamW updates in fp32, downcast to bf16 only for the model’s forward/backward pass. After the fix, all previously-frozen parameters began moving immediately.

A second gradient instability was diagnosed and fixed: the retention layer’s γ parameter drifted to 1.0 (no decay), causing its $O(T^2)$ gradient to dominate the global gradient clip and starve all other parameters of update signal. The fix combines three techniques: (1) clamping $\log \gamma \leq -0.02$ ($\gamma \leq 0.98$) to ensure the decay matrix remains contractive, (2) a $0.01\times$ LR group for γ to compensate for the gradient scale mismatch, and (3) skip-on-spike to skip optimizer steps when the pre-clip gradient norm exceeds 500. After these fixes, all three cells train stably from scratch with gradient norms in the range 1–18 and no explosions through the first 150 steps.

Early evaluation results (5 examples per task per depth, depths 2/4/6/8):

Cell	Step	Task 1 (chain)	Task 2 (2-var)	Task 3 (free-form)	Overall
A (control)	2900	0%	95%	100%	65%
B (workspace)	400	0%	25%	5%	10%
C (full)	200	5%	25%	0%	10%

The evaluation results above are from the previous run (before the γ stability fix), retained as the only available evaluation data. The current fresh run is too early for meaningful evaluation. Cell A (the no-workspace control) was performing as expected: near-perfect on Tasks 2 and 3, but cannot do multi-hop chain reasoning (Task 1). Cells B and C were still early (4–8% of the

training budget) but training stably. It is too early to assess whether the workspace provides a Task 1 benefit.

Backward pass verification. The workspace backward pass has been verified against PyTorch autograd on CPU. All gradients—input gradients, weight gradients (8 matrices), slot gradients, norm weights, and gate scalars—match to within float32 precision ($< 10^{-4}$ relative error).

8 Discussion

8.1 Why This Might Work

The J-SPACE finding tells us that reasoning in large models is carried by a small, privileged subspace. The TRM finding tells us that iteration through a small network can substitute for parameter count. The retention coincidence tells us that the retention layer’s compressed state already has the functional shape of the J-SPACE. Jasper combines all three: the workspace provides the privileged subspace, the recurrent core provides the iteration, and the hybrid retention backbone provides the substrate.

If this works, the implication is that we can build reasoning models that are:

- **Small enough for the browser:** $\sim 1\text{B}$ parameters, flat memory in both sequence length and loop count.
- **Controllable at inference:** K can be adjusted per problem, trading compute for accuracy.
- **Interpretable:** workspace probes (R3) provide a window into the model’s reasoning, unlike opaque latent reasoning in recurrent transformers.
- **Verifiable:** selective ablation (R4) provides a causal test of whether the workspace is doing the reasoning, not just correlated with it.

8.2 Why It Might Not Work

Synthetic-task wins at 30M parameters have a real history of not transferring to language at scale. The tasks we use (arithmetic chains, permutation tracking) are structurally simple and may not capture the full complexity of language reasoning. The workspace may help on these tasks without generalizing to the kind of reasoning that matters for real applications.

The gradient stabilization techniques consumed most of our engineering effort, and we should be clear that little of what resolved them was novel. Stacking K residual updates accumulates gradient as $\sqrt{K}$, so $1/\sqrt{K}$ scaling is arithmetic; a parameter inside a positive feedback loop needs its magnitude constrained, so slot normalization is routine; unbounded attention logits call for spectral normalization, standard since Miyato et al. (2018); an additive module should begin at identity, standard since LoRA; and a recurrent state with no forget term will not stay bounded, which LSTMs settled in 1997. The capacity mismatch at $C = 1$ is likewise a closed-form consequence—bounding the spectral norm at C caps attention logits at $C^2/\sqrt{d_h}$, giving a ratio of $\approx 1.2:1$ at $C = 1$, which is uniform to within noise—and could have been computed before the run rather than diagnosed after it. These properties belong in the initial design, and we present them in Section 6.2 as design requirements derived from the architecture rather than as findings.

What was not obvious, and what we got wrong, is the measurement. We initially adopted attention entropy as a proxy for selectivity and optimized it directly. This is a measurement error: entropy scores peakedness rather than input-dependence, and the cheapest way to minimize entropy is a fixed routing that ignores the input—precisely the degenerate solution the regularizer was intended to prevent. The general lesson—that a proxy admitting a degenerate optimum will find it, and that “sharp” must be distinguished from “informative”—transfers beyond this architecture, and it is the part of this work we would defend.

The more substantive question is whether the workspace provides enough *computational leverage* to matter. The current run is the first clean test: all stabilizations are applied from the start, the optimizer bug that froze the workspace parameters has been fixed, and the 10,000-step budget gives the workspace time to open its gates and demonstrate (or fail to demonstrate) a Task 1 benefit.

Two possibilities remain open. The first is that our experimental design tests the wrong thing: Task 1 requires iteration *and* memory, and Cell B supplies memory without iteration, so B-versus-A may be architecturally incapable of showing a benefit regardless of workspace quality. On this reading the informative comparison is C-versus-B, and we have under-prioritized C because it is slower. The second is that the workspace as designed is simply not the right mechanism. It requires eight weight matrices, two gates, a decay term, two normalizations, and three learning-rate groups to train at all; that quantity of scaffolding is itself evidence against the design, since the architectures that won in practice—transformers included—won by being simple and trainable rather than by being optimal. Accordingly we have set an advance stopping rule: if Cell C shows Task 1 $\leq 10\%$ by step 2,000 with gates verifiably open and attention verifiably input-dependent, we report the negative result rather than continue to search for one more fix.

Finally, the retention coincidence may be more metaphorical than functional. The retention state and the J-SPACE have similar shapes, but they operate at different levels of abstraction—the retention state is a low-level sequence summary, while the J-SPACE is a high-level conceptual workspace. Whether the workspace module can bridge this gap is an empirical question.

8.3 Decision Framework

The proof-of-concept is designed to produce a clear go/no-go decision before any significant cloud spend:

- **Go:** R1 and R2 pass, and at least one of R3/R4 shows the workspace doing real causal work $\rightarrow$ fund a 300M language-scale ablation ($\sim \$3\text{--}5\text{K}$).
- **Pivot:** R1 fails but Cell A’s efficiency story stands $\rightarrow$ keep the hybrid backbone, drop the workspace, proceed on hybrid distillation.
- **Kill:** Cell C $\leq$ Cell A everywhere $\rightarrow$ the novel architecture track is dead; fall back to sampled-attempts-plus-verifier on a standard model.

The entire proof-of-concept costs $\sim \$20\text{--}30$ of electricity and under a week of compute. This buys the right to spend $\$5\text{K}$ intelligently, not the conclusion itself.

9 Related Work

Global workspace in LLMs. Anthropic (2026) discovered the J-SPACE in Claude Opus 4.6 using the Jacobian lens, showing that a small set of verbalizable representations functions as a global workspace (Baars, 2005; Dehaene, 2014). Our workspace module is an explicit engineering of this concept.

Iterative and recurrent reasoning. Jolicoeur-Martineau (2025) showed that tiny recursive networks (7M parameters) can outperform much larger models on ARC-AGI through iterative refinement. Geiping et al. (2025) scaled recurrent-depth transformers to 3.5B parameters, demonstrating test-time compute scaling in latent space. Gehring et al. (2024) explored relaxed recursive transformers with shared parameters. Our recurrent core follows the TRM paradigm but uses a hybrid retention/attention backbone and an explicit workspace state.

Retention and hybrid models. Sun et al. (2024) introduced the Retentive Network, replacing softmax attention with decayed linear attention that maintains a fixed-size recurrent state. Hybrid

models that combine recurrent-state layers with attention have been shown to match transformer-based R1-distilled models on AIME/MATH while generating $3\times$ faster, and NVIDIA’s Nemotron line has repeatedly shown hybrid models matching or beating same-size transformers on long reasoning traces. Our architecture follows the hybrid pattern but adds the workspace and recurrent core.

Test-time compute scaling. The broader literature on scaling test-time compute—from self-consistency (Wang et al., 2023) to process reward models (Lightman et al., 2023) to OpenAI’s o1/o3 models—operates in token space. Our approach scales test-time compute in latent space, which is more memory-efficient but less legible.

10 Conclusion

We have presented Jasper, an architecture that combines three recent insights—the J-SPACE as a compact reasoning workspace, iterative refinement as a substitute for parameter count, and the retention layer’s compressed state as a native substrate for the workspace—into a single system. The architecture reasons in latent space without generating tokens, keeps memory flat in both sequence length and loop count, and provides interpretable and causally testable reasoning through workspace probes and selective ablation. The retention backbone (decayed linear attention) preserves the key properties of a fixed-size, input-dependent state while being simple and fast on the target hardware.

The proof-of-concept experiment is designed to produce a clear verdict at minimal cost: if the workspace causally carries multi-step reasoning (R4) and the recurrent core provides test-time compute scaling (R2), the architecture warrants scaling to language-scale experiments. If not, the hybrid retention-attention backbone still stands as an efficient baseline for in-browser deployment.

The broader claim is that reasoning in language models need not be tied to token generation. The brain reasons through recurrent firing patterns before articulating a thought; perhaps language models should too.

References

- Wes Gurnee and Jack Lindsey et al. Verbalizable representations form a global workspace in language models. *arXiv preprint arXiv:2607.15495*, 2026.
- Takeru Miyato, Toshiaki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Alexandre Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025. [Samsung SAIT Montreal]
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Bernard J. Baars. Global workspace theory of consciousness: Toward a neuroscience of human experience. *Progress in Brain Research*, 150:45–53, 2005.
- Stanislas Dehaene. *Consciousness and the Brain: Deciphering How the Brain Codes Our Thoughts*. Viking, 2014.
- Simran Arora, Sabhash Rao, and Christopher Ré. Theoretical limitations of self-attention and state space models in reasoning tasks. *arXiv preprint*, 2024.

- Jonas Gehring, Kilian Golde, and David Grangier. Relaxed recursive transformers. *arXiv preprint arXiv:2410.10672*, 2024. [Google DeepMind]
- Xuezhi Wang et al. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Hunter Lightman et al. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. [OpenAI]
- Hongyu Wang et al. DeepNet: Scaling transformers to 1,000 layers. *arXiv preprint arXiv:2203.00555*, 2022. [Microsoft]
- Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Zhangyin Feng, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2024. [Microsoft]